Тема 2. Базовые типы данных языков программирования высокого уровня

Лекция 3

Базовые типы данных языков программирования высокого уровня (часть 1)

Любой набор знаков, рассматриваемый безотносительно к его содержательному смыслу, называют **данными.**

Данные обычно изображают некоторую информацию, которую можно получить, если известен **смысл**, приписываемый данным. Однако, в программировании, особенно в системном, часто приходится **иметь дело именно с данными**. Например, разрабатывая систему хранения и поиска некоторых текстов, программист может не знать их содержания. Его задача - **обеспечить экономное использование памяти** и быстрый поиск требуемых текстов по заданным признакам. Для решения этой задачи достаточно знать лишь количественные характеристики текстов, рассматриваемых как данные. Вообще вычислительные машины выполняют только обработку данных, которая заинтересованным лицам, приписывающим этим данным некоторый смысл, представляется обработкой информации.

Совокупности данных, организованные некоторым образом, называют **структурами данных**. Структура определяется отношениями между ее элементами. В рамках данного курса будем изучать структуры данных: стек, очередь, массивы, списки, деревья и таблицы.

Информация по каждому типу однозначно определяет:

- структуру хранения данных указанного типа, то есть выделение памяти, представление данных в ней и метод доступа к данным;
- множество допустимых значений, которые может иметь тот или иной объект описываемого типа;

- набор допустимых операций, которые применимы к объекту описываемого типа.

Абстрактный тип данных (АТД) – это формализованное описание (модель), определяющее организацию и набор возможных операций с описываемыми данными.

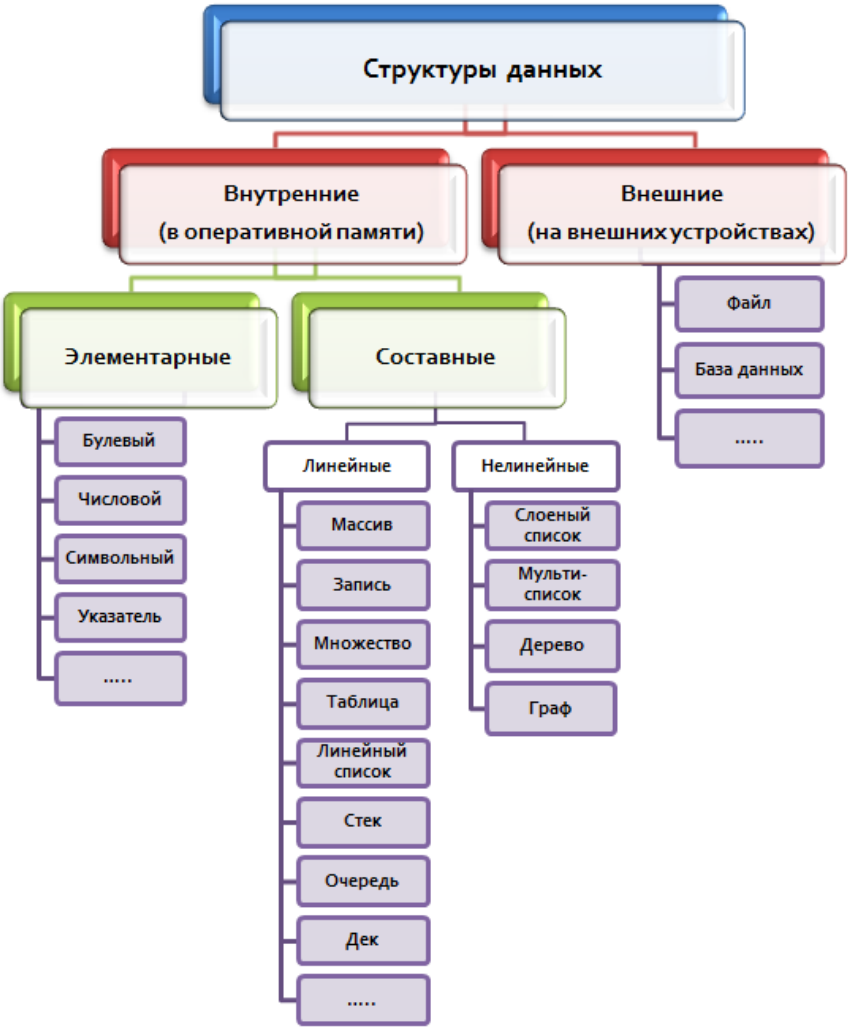


Рис. 2.1. Классификация структур данных в зависимости от размещения физических структур и доступа к ним



Рис. 2.2. Классификация базовых и дополнительных структур данных

Компьютерные данные это дискретные сообщения, которые представлены в форме, используемой в компьютере, *понятной компьютеру*. Для процессора компьютера любые данные представляют собой ***неструктурированную*** последовательность битов (иногда используют термин поток битов).

Конкретная интерпретация этой последовательности зависит от программы, от *формы представления и структуры данных*, которые выбраны *программистом*. Это выбор, в конечном счёте, зависит от решаемой задачи и удобства выполнения действий над данными.

К данным в программах относятся:

- ***Непосредственные значения*** - это *неизменные* объекты программы, которые представляют сами себя: числа (25, 1.34E-20), символы (‘A’, ‘!’), строки (‘Введите элементы матрицы’);
- ***Константы*** – это имена, закрепляемые за некоторыми значениями (const pi=3.1415926);

- ***Переменные*** - это объекты, которые могут принимать значение, сохранять его без изменения, и изменять его при выполнении определенных действий (var k:integer, x:real, a:array[1..3,1..5]);

- ***Значения выражений и функций***. Выражения и функции – это записанные определённым способом правила вычисления значений: k*x+ sqrt (x).

Для отображения особенностей представления в компьютере данных различной природы в информатике, в компьютерных дисциплинах используется важнейшая *концепция типа данных.*

Наиболее часто используемые **предопределенные скалярные** типы: целый (***integer***), вещественный (***real***), символьный (***char***), логический (***boolean***).

Тип ***integer.*** Целочисленные точные значения. Примеры: **73, -98, 5, 19674.** Машинное представление: формат с фиксированной точкой. Диапазон значений определяется длиной поля. Операции: **+, -, *, div, mod,=, <,** и т.д.

Тип ***real.*** Нецелые приближенные значения. Примеры: **0.195, -91.84, 5.0.** Машинное представление: формат с плавающей точкой. Диапазон и точность значений определяется длиной поля. Операции: **+, -, *,** /, **=, <,** и т.д.

Тип ***char.*** Одиночные символы текстов. Примеры: **‘a’, ‘!’, ‘5’.** Машинное представление: формат ASCII. Множество значений определяется кодовой таблицей и возможностями клавиатуры. Операции: **+, =, <,** и т.д.

Тип ***boolean.*** Два логических значения **false** и **true**. Причем, **false<true.** Машинное представление ─ нулевое и единичное значение бита: false кодируется 0, true ─ 1. Операции: **¬, ∨, ∧, =,** < и т.д.

Типы данных, встроенные в язык программирования высокого уровня, являются базой для конструирования производных структур. Поэтому для понимания организации производных структур на логическом и физическом уровнях важно иметь четкое представление об организации базовых типов данных. ***Целью данной темы*** является рассмотрение вопросов представления базовых типов данных в языках программирования. В ней

описываются физический уровень представления данных в компьютере, простые базовые типы данных и поддерживаемые современными языками составные структуры данных, такие как массивы, строки, структуры (записи) и множества.

2.1. Основы организации данных на физическом уровне

В отличие от абстрактного уровня, логический и физический уровни представления структур данных зависят от конкретного языка программирования и ЭВМ, на которой исполняется программа. Логический и физический уровни отличаются друг от друга, поэтому в вычислительных системах осуществляется отображение физического уровня на логический и наоборот.

Рассмотрим основы организации данных на физическом уровне. Носителем информации, обрабатываемой в компьютере, является специальное устройство, называемое ***оперативной памятью***. Минимальной единицей информации в оперативной памяти является ***бит***. Бит может быть выключен, так что его значение есть нуль, или включен, тогда его значение равно единице.

Единственный бит не может представить много информации в отличие от группы битов. Группа (последовательность) из восьми битов представляет собой ***байт***.

Оперативная память устроена так, что доступ к хранимой в ней информации осуществляется побайтно. Таким образом, оперативную память можно рассматривать как конечную последовательность байт. Номер байта в этой последовательности называется ***физическим адресом*** ячейки памяти (или просто ***адресом***). Физический адрес используется для получения доступа к конкретной ячейке.

Следует отметить, что байт является минимально адресуемой ячейкой памяти, но существуют и другие, более крупные ячейки, представляющие собой последовательности байт, расположенных в памяти друг за другом. Обычно выделяют следующие типы ячеек памяти: ***байт***, ***слово*** и ***двойное слово***.

- ***Байт*** – восемь последовательно расположенных битов, пронумерованных от 7 до 0, при этом бит 0 является самым младшим значащим битом;

- ***Слово*** – последовательность из двух байт, имеющих последовательные адреса. Размер слова – 16 бит; биты в слове нумеруются от 15 до 0. Байт, содержащий нулевой бит, называется младшим байтом, а байт, содержащий 15-й бит, – старшим байтом. Адресом слова считается адрес его младшего байта. Адрес старшего байта может быть использован для доступа к старшей половине слова.

- ***Двойное слово*** – последовательность из четырех байт (32 бита), расположенных по последовательным адресам. Нумерация этих бит производится от 31 до 0. Слово, содержащее нулевой бит, называется младшим словом, а слово, содержащее 31-й бит, - старшим словом. Младшее слово хранится по меньшему адресу. Адресом двойного слова считается адрес его младшего слова. Адрес старшего слова может быть использован для доступа к старшей половине двойного слова.

Количество бит составляющих ячейку памяти называется ее ***разрядностью***.

Кроме трактовки ячеек памяти с точки зрения их разрядности, процессор ЭВМ на уровне команд поддерживает логическую интерпретацию этих типов. С точки зрения логической интерпретации выделяют следующие типы данных на физическом уровне: ***целые числа без знака, целые числа со знаком, вещественные числа*** различной точности, ***указатель, цепочка*** байт, ***символ*** и ***строка***.

Целые числа без знака – двоичное значение без знака, размером 8, 16 или 32 бита. Основой этого представления является запись целого без знакового числа в двоичной системе счисления, где каждый бит соответствует двоичной цифре.

Числовой диапазон для этого типа следующий:

- байт – от 0 до $2^8 – 1 = 255$;
- слово – от 0 до $2^{16} – 1 = 65\,535$;
- двойное слово – от 0 до $2^{32} – 1 = 4\,294\,967\,295$.

Целые числа со знаком – двоичное значение со знаком, размером 8, 16 или 32 бита. Знак в этом двоичном числе содержится в 7, 15 или 31-м бите соответственно. Ноль в этих битах в операндах соответствует положительному числу, а единица – отрицательному. Целые числа со знаком представляются в дополнительном коде.

Дополнительный код числа предполагает фиксированное количество разрядов (обозначим его символом n) и образуется путем сложения соответствующего числа с числом 2^n с последующим взятием остатка от деления на 2^n. В этом случае результат операции называется дополнением исходного числа до 2^n. Рассмотрим пример представления целых чисел со знаком в дополнительном коде. Пусть разрядность чисел составляет 8 бит. Число до которого будет строится дополнение есть $2^8=256$. Положительное число 3 в дополнительном коде есть $(256+3)\ mod\ 256 = 3 = 00000011_2$. Отрицательное число –3 в дополнительном коде есть

$(256–3)\ mod\ 256 = 253 = 11111101_2$.

Здесь и далее символ *mod* есть операция взятия остатка от деления.

Для облегчения представлений чисел в дополнительном коде существует следующее правило:

1. если число положительное, то его двоичная запись из n разрядов совпадает с дополнительным кодом;

2. если число отрицательное, то необходимо записать модуль этого числа в двоичной системе счисления с использованием n разрядов, затем каждый бит этой записи проинвентировать

(заменить нулевые биты на единичные и наоборот) и к полученному результату прибавить единицу.

Например, модуль числа –3 в двоичном виде из 8 цифр есть 00000011_2. После инвертирования получим 11111100_2. После прибавления единицы – 11111101_2, что соответствует представлению числа –3 в дополнительном коде (см. предыдущий пример).

Числовые диапазоны для представления целых чисел со знаком следующие:

- 8-разрядное целое – от – 128 до 127;
- 16-разрядное целое – от – 32 768 до 32 767;
- 32-разрядное целое – от -2^{31} до $2^{31} - 1$.

Вещественные числа на физическом уровне представляются в формате с плавающей точкой, который предполагает кодирование чисел в виде трех составляющих: знака, мантиссы и порядка. В соответствии со стандартом IEEE 754 (Standard Floating Point Representations) существуют три формата: 32-битный, 64-битный и 80-битнй форматы представления чисел с плавающей точкой.

32-битный формат называется форматом с одинарной точностью. Вещественные числа в этом формате занимаю 4 байта, из которых старший бит кодирует знак числа, следующие за ним 8 бит – порядок числа и, наконец оставшиеся 23 бита – мантиссу. Этот формат представлен на рис. 2.3.

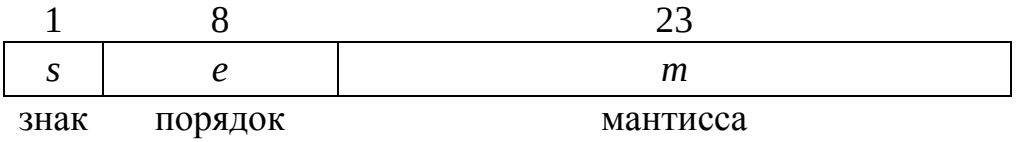


Рис. 2.3. Формат представления числа с плавающей точкой одинарной точности

Формула для вычисления значения числа для этого представления:

$$v = \begin{cases} (-1)^s \cdot 2^{e-127} \cdot 1.m, & \text{если } 0 < e < 255; \\ (-1)^s \cdot 2^{126} \cdot 0.m, & \text{если } e = 0 \text{ и } m \neq 0; \\ (-1)^s \cdot 0, & \text{если } e = 0 \text{ и } m = 0; \\ (-1)^s \cdot Inf, & \text{если } e = 255 \text{ и } m = 0; \\ NaN, & \text{если } e = 255 \text{ и } m \neq 0. \end{cases} \quad (2.1)$$

В этой формуле символом *Inf* обозначена бесконечность, а символом *NaN* – нечисловое значение.

В этом формате представимы вещественные числа, входящие в диапазон от $1.5 \cdot 10^{-45}$ до $3.4 \cdot 10^{+38}$. Здесь и далее диапазон указан без учета знака числа, то есть самым маленьким отрицательным числом может быть $-3.4 \cdot 10^{+38}$, а самым маленьким положительным – число $1.5 \cdot 10^{-45}$, и наоборот – самое большое отрицательное число: $-1.5 \cdot 10^{-45}$, а самое большое положительное – число $3.4 \cdot 10^{+38}$.

64-битный формат называется форматом с двойной точностью. Вещественные числа в этом формате занимаю 8 байт, из которых старший бит кодирует знак числа, следующие за ним 11 бит – порядок числа и, наконец оставшиеся 52 бита – мантиссу. Этот формат представлен на рис. 2.4.



Рис. 2.4. Формат представления числа с плавающей точкой двойной точности

Формула для вычисления значения числа для этого представления:

$$v = \begin{cases} (-1)^s \cdot 2^{e-1023} \cdot 1.m, \text{ если } 0 < e < 2047; \\ (-1)^s \cdot 2^{1022} \cdot 0.m, \quad \text{если } e = 0 \text{ и } m \neq 0; \\ (-1)^s \cdot 0, \qquad \text{если } e = 0 \text{ и } m = 0; \\ (-1)^s \cdot Inf, \qquad \text{если } e = 2047 \text{ и } m = 0; \\ NaN, \qquad \text{если } e = 2047 \text{ и } m \neq 0. \end{cases} \quad (2.2)$$

В этом формате представимы вещественные числа, входящие в диапазон от $5.0 \cdot 10^{-324}$ до $1.7 \cdot 10^{+308}$.

80-битный формат называется расширенным форматом с двойной точностью. Вещественные числа в этом формате занимаю 10 байт, из которых старший бит кодирует знак числа, следующие за ним 15 бит – порядок числа и, наконец оставшиеся 63 бита – мантиссу. Этот формат представлен на рис. 2.5.

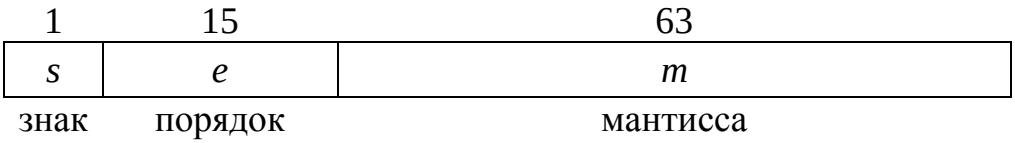


Рис. 2.5. Расширенный формат представления числа с плавающей точкой двойной точности

Формула для вычисления значения числа для этого представления:

$$v = \begin{cases} (-1)^s \cdot 2^{e-16383} \cdot 1.m, \text{ если } 0 < e < 32767; \\ (-1)^s \cdot 2^{16382} \cdot 0.m, \quad \text{если } e = 0 \text{ и } m \neq 0; \\ (-1)^s \cdot 0, \qquad \text{если } e = 0 \text{ и } m = 0; \\ (-1)^s \cdot Inf, \qquad \text{если } e = 32767 \text{ и } m = 0; \\ NaN, \qquad \text{если } e = 32767 \text{ и } m \neq 0. \end{cases} \quad (2.3)$$

В этом формате представимы вещественные числа, входящие в диапазон от $3.4 \cdot 10^{-4932}$ до $1.1 \cdot 10^{+4932}$.

Указатель – целочисленное значение, содержащее адрес в оперативной памяти.

Цепочка представляет собой некоторый непрерывный набор байтов, или слов максимальной длиной до 64 Кбайт.

Символ – байт, в который записывается код символа – целое от 0 до 255. В ЭВМ используется система кодировки ASCII (American Standard Code for Information Interchange).

Строка – последовательность символов, которая размещается в соседних байтах памяти, так, что код первого символа строки записывается в первом байте, код второго символа – во втором байте и т.п. Адресом строки считается адрес ее первого байта.

2.2. Классификация базовых типов и структур данных

Современные языки программирования высокого уровня имеют довольно обширный набор базовых типов и структур данных и, который может изменяться от языка к языку. Они обеспечивают удобные средства работы с данными на абстрактном, и, прежде всего на логическом уровне, при этом практически скрывая особенности представления на физическом уровне.

Можно приводить различные классификации типов данных, например, простые и составные типы, предопределенные и определяемые типы и т.д. Существенно то, что, несмотря на многолетнее использование типов данных в отечественном программировании, так и не сложилась устойчивая и общепринятая русскоязычная терминология. Поэтому будем использовать некоторый набор терминов, выбранных из соображений максимальной распространенности и интуитивной ясности.

Выделим следующие категории типов.

1. ***Встроенные типы данных***, т.е. типы, предопределенные в языке программирования. Обычно в языке фиксируются внешнее представление значений этих типов (вид литеральных констант) и набор операций с описанием их семантики. Внутреннее представление и реализация операций выбираются в конкретных компиляторах и подсистемах поддержки выполнения программ.

2. Под термином "***уточняемый тип данных***" понимается возможность определения типа на основе встроенного типа данных, значения которого упорядочены. В частности, к категории уточняемых типов относится тип поддиапазона целых чисел в языках линии Паскаль.

3. Категорию ***перечисляемых типов данных*** составляют явно определяемые целые типы с конечным числом именованных значений. Это очень простой и легко реализуемый механизм, часто являющийся очень полезным.

4. ***Конструируемые типы*** (иногда их называют составными) обладают той особенностью, что в языке предопределены средства спецификации таких типов и некоторый набор операций, дающих возможность доступа к компонентам составных значений. Примерами наиболее распространенных разновидностей конструируемых типов являются массивы, строки, структуры (записи) и множества.

5. ***Указательные типы*** дают возможность работы с типизированными множествами абстрактных адресов переменных, содержащих значения некоторого типа. В сильно типизированных языках (Паскаль, Модула, Ада и т.д.) работа с указателями сильно ограничена. В частности, невозможно получить значение указателя явно определенной переменной и/или применять к известным значениям указателей адресную арифметику. В языках с более слабой типизацией (например, Си/C++) допускаются практически неограниченные манипуляции указателями.

Вообще говоря, упоминавшиеся выше уточняемые, перечисляемые и конструируемые типы данных являются типами, определяемыми пользователями. Но эти определения не могут включать спецификацию операций над значениями типов. Допустимые операции либо

предопределены, либо наследуются от некоторого определенного ранее или встроенного типа. Под термином "***определяемый пользователем тип данных***", будем понимать возможность полного определения нового типа, включая явную или неявную спецификацию множества значений, спецификацию внутреннего представления значений типа и спецификацию набора операций над значениями определяемого типа.

Наконец, под термином "***полнотиповая система***" будем понимать систему типов, в которых типы, определяемые пользователем, равноправны с предопределенными типами, т.е. можно, например, определить тип массива с элементами любого определенного типа, можно использовать определяемый пользователем тип на основе любого определенного типа и т.д.

2.3. Встроенные типы данных

Обычно в состав встроенных типов данных включаются такие типы, операции над значениями которых напрямую или, по крайней мере, достаточно эффективно поддерживаются командами компьютеров. В современных компьютерах к таким "машинным" типам относятся целые числа разного размера (от одного до восьми байт), булевские значения (поддерживаемые обычно за счет наличия признаков условной передачи управления) и числа с плавающей точкой одинарной и двойной точности (обычно четыре и восемь байт соответственно).

В соответствии с этим, в традиционный набор встроенных типов обычно входят следующие типы:

Тип CHARACTER (или CHAR) в разных языках – это:

1) либо набор печатных символов из алфавита, зафиксированного в описании языка (для большинства языков англоязычного происхождения этот алфавит соответствует кодовому набору ASCII);

2) либо произвольная комбинация нулей и единиц, размещаемых в одном байте.

В первой интерпретации (свойственной языкам линии Паскаль) для значений типа CHAR определены только операции сравнения в соответствии

с принятым алфавитом. Например, при использовании ASCII выполняются соотношения '0' < '1' < ...< '9' < 'A' < 'B' < ...< 'Z' < 'a' < 'b' < ...< 'z'; известно, что если значение переменной *x* удовлетворяет условию '0' < *x* < '9', то это значение – цифра; если 'A' < *x* < 'Z', то значение *x* – прописная буква; если 'a' < *x* < 'z', то значение *x* – строчная буква и т.д. При использовании этой интерпретации арифметические операции над символьными значениями не допускаются.

Во второй интерпретации (свойственной языкам линии Си) литеральными константами типа *CHAR* по-прежнему могут быть печатные символы из принятого в языке алфавита, но возможно использование и числовых констант, задающих желаемое содержимое байта. В этом случае, как правило, над значениями типа *CHAR* возможно выполнение не только операций сравнения, но и операций целочисленной арифметики.

Наконец, в некоторых языках явно различают тип *CHAR* как чисто символьный тип и тип сверхмалых целых (*TINY INTEGER*) как тип целых чисел со значениями, умещающимися в один байт.

В современных компьютерах, как правило, поддерживается целочисленная байтовая арифметика, обеспечивающая как первую, так и вторую интерпретацию типа CHAR.

Тип ***BOOLEAN*** (или ***BOOL***) в тех языках, где он явно поддерживается, содержит два значения – *TRUE* (истина) и *FALSE* (ложь). Несмотря на то, что для хранения значений этого типа теоретически достаточно одного бита, обычно в реализациях переменные этого типа занимают один байт памяти. Для всех типов данных, для которых определены операции сравнения, определены также и правила, по которым эти операции сравнения вырабатывают булевские значения. Над булевскими значениями возможны операции конъюнкции (&& или AND), дизъюнкции (|| или OR) и отрицания (~ или NOT), определяемые следующими таблицами истинности:

TRUE AND TRUE =TRUE;

TRUE AND FALSE = FALSE;

FALSE AND TRUE = FALSE;

FALSE AND FALSE = FALSE;

TRUE OR TRUE = TRUE;

TRUE OR FALSE = TRUE;

FALSE OR TRUE = TRUE;

FALSE OR FALSE = FALSE;

NOT FALSE = TRUE;

NOT TRUE = FALSE.

В языках линии Си прямая поддержка булевского типа данных отсутствует, но имеется логическая интерпретация значений целых типов. Значением операции сравнения может быть "0" (*FALSE*) или "1" (*TRUE*). Значение целого типа "0" интерпретируется как *FALSE*, а значения, отличные от нуля, - как *TRUE*. В остальном все работает как в случае наличия явной поддержки булевского типа.

Тип целых чисел в общем случае включает подмножество целых чисел, определяемое числом разрядов, которое используется для внутреннего представления значений. При определении типа целых чисел обычно стремятся к тому, чтобы множество его значений было симметрично относительно нуля (собственно, это стимулируется и стандартными свойствами машинной целочисленной арифметики). Поэтому приходится тратить один бит на значение знака числа и при использовании n бит для внутреннего представления целого соответствующий тип содержит значения в диапазоне от $-2n\text{-}1$ до $2n\text{-}1$. В подавляющем большинстве современных процессоров отрицательные целые числа обычно представляют в дополнительном коде.

В языках, ориентированных на 32-разрядные компьютеры, в частности, в стандартных Си и C++ для рационального использования памяти допускаются модификации целого типа *short* (обычно 16-разрядные), *int* (обычно то же самое, что и *long*) и *long* (обычно 32-разрядные), а также байтовые целые (*char*). При этом поддерживаются автоматические преобразования значений типов меньшего размера к значениям типов большего размера.

Наряду со знаковыми целыми типами в языках часто поддерживаются ***беззнаковые целые***. Такие типы в линии языков Паскаль называются *CARDINAL*, а в линии языков Си именуются путем добавления модификатора *unsigned* к названию соответствующего целого типа. Таким образом, в последнем случае существуют типы *unsigned char*, *unsigned short*, *unsigned int* и *unsigned long*. Поскольку множество значений типа *unsigned* в два раза мощнее множества значений соответствующего целого типа, то поддерживается их автоматическое преобразование только к целым типам большего размера.

Наконец, для поддержки численных вычислений в языках обычно специфицируется встроенный ***тип чисел с плавающей точкой*** с базовым названием *REAL* или *FLOAT*. Обычно в описании языков не фиксируется диапазон и точность значений такого типа. Они уточняются в реализации и обычно существенно зависят от особенностей целевого процессора. В языках семейства Си (32-разрядных) специфицированы три разновидности типа чисел с плавающей точкой – *float* (обычно с размером 32 бит), *double* (размером в 64 бит) и *long double* (размером 80 бит), которым на физическом уровне соответствуют форматы представления стандарта IEEE 754.

2.4. Уточняемые типы данных

Основная идея уточняемого типа данных состоит в том, что для любого значения любого встроенного (и перечисляемого) типа существует его внешнее литеральное представление. Более того, по литеральному представлению константы можно однозначно определить тип, к которому она относится. Если к тому же на множестве значений типа задано отношение порядка (определены операции сравнения), то иногда возникает

потребность сказать, что в данном приложении нас интересует подмножество значений такого типа, ограниченное некоторым специфицированным диапазоном. По причине наличия упорядоченности значений такой диапазон может быть задан парой литеральных констант базового типа *c1* и *c2*, удовлетворяющих условию $c1 < c2$. Тем самым, определение нового уточненного типа может иметь вид, пример из языка Модула-2:

TYPE T = [c1..c2].

Основной проблемой уточняемых типов является потребность в динамическом контроле значений, формируемых при вычислении выражений и возвращаемых функциями. Если для значений базовых типов (по крайней мере, числовых) такой контроль, как правило, поддерживается аппаратурой компьютера, то для уточняемых типов, вообще говоря, требуется программный контроль, вызывающий серьезные накладные расходы. В развитых компиляторах обычно поддерживаются два режима компиляции – отладочный со всеми возможными контролирующими действиями во время выполнения программы и "рабочий", в котором контроль отключается. Однако, если учесть, что в любой серьезной программе ошибки сохраняются на протяжении всей ее жизни, бесконтрольное выполнение программ очень затрудняет нахождение таких ошибок.

Следует отметить, что линии языков Си уточняемые типы данных не предусмотрены.

2.5. Перечисляемые типы данных

Перечисляемый тип состоит из конечного числа упорядоченных именованных значений. В классическом варианте, свойственном, например, языкам линии Паскаль, определение типа состоит из перечисления имен значений (поэтому справедливо называть такой тип перечисляемым), эти имена в дальнейшем играют роль имен литеральных констант этого типа и должны отличаться от литерального изображения констант любого другого типа. Поскольку значения типа задаются путем перечисления, каждому значению можно однозначно сопоставить натуральное число от 1 до n, где n – число значений перечисляемого типа.

Обычно для любого перечисляемого типа предопределяются операции получения значения по его номеру и получения номера по значению. Кроме того, для перечисляемого типа предопределяются операции сравнения и получения следующего и предыдущего значения. По причине однозначного сопоставления значению перечисляемого типа натурального числа, возможно неявное преобразование этих значений к значению любого числового типа данных.

В языках линии Си под тем же термином "перечисляемый тип" понимается нечто другое, поскольку при определении такого типа можно явно сопоставить имени значения некоторое целое (не обязательно положительное) число. При отсутствии явного задания целого первому элементу перечисляемого типа неявно соответствует 0, а каждому следующему – целое значение, на единицу большее целого значения предыдущего элемента.

При этом:

a) использование имени перечисляемого типа для объявления переменной эквивалентно использованию типа *int*, и такая переменная может содержать любое целое значение;

b) имена значений перечисляемого типа на самом деле понимаются как имена целых констант, и к этим значениям применимы все операции над целыми числами, даже если они выводят за пределы множества целых значений элементов перечисляемого типа.

Так что перечисляемый тип в смысле языков линии Си – это не совсем тип в строгом смысле этого слова, а скорее удобное задание группы именованных констант целого типа. Проиллюстрируем сказанное примером использования перечисляемых типов в языке C++.

На логическом уровне перечисляемый тип (или просто перечисление) определяются с помощью ключевого слова *enum*, которое указывает на начало перечисляемого типа. Стандартный вид определения перечислений следующий:

enum ярлык {список перечислений} список переменных;

Как имя перечисления – ярлык, так и список переменных необязательны, но один из них должен присутствовать. Список перечислений – это разделенный запятыми список идентификаторов (допустимых значений перечисления). Имя перечисления используется для объявления переменных данного типа. Следующий фрагмент определяет перечисление **coin** (монета) со списком монет в США: *penny* (один цент), *nickel* (пять центов), *dime* (десять центов), *quarter* (двадцать пять центов), *half_dollar* (полдоллара), *dollar* (доллар) и объявляет переменную *money* этого типа:

```
enum coin {penny, nickel, dime, quarter, half_dollar, dollar};

enum coin money;
```

Имея данное определение и объявление, следующий тип присваивания совершенно корректен:

```
money = dime;

if (money == quarter) printf("is a quarter\n");
```

Значением перечисления на физическом уровне является целое число – порядковый номер идентификатора значения логического уровня в списке. Нумерация начинается с 0.

Так для нашего примера значениям логического уровня представления соответствуют следующие значения физического уровня:

penny	0
nickel	1
dime	2
quarter	3
half_dollar	4
dollar	5

В C++ имеется возможность определить значения одного или нескольких символов, используя инициализатор. Это делается путем

помещения за символом знака равенства и целочисленного значения. При использовании инициализатора, символы, следующие за инициализационным значением, получают значение большее, чем указанное перед этим. Например, в следующем объявлении *quarter* получает значение 100.

enum coin { penny, nickel, dime, quarter=100, half_dollar, dollar};

Теперь эти символы получат следующие значения на физическом уровне:

penny	0
nickel	1
dime	2
quarter	100
half_dollar	101
dollar	102

Размерность значений перечисления на физическом уровне определятся по максимальному числовому значению: если оно меньше чем 256, то для представления в памяти отводится один байт, а в противном случае – два.

Структуры и алгоритмы компьютерной обработки данных: Учебно-практическое пособие / Сост. В.В. Румбешт. – Белгород: Изд-во БелГУ, 2007

Приложение

Ключарев А. А., Матьяш В. А., Щекин С. В.
Структуры и алгоритмы обработки данных: Учеб. пособие/
СПбГУАП. СПб., 2003. 172 с.

1. Элементарные данные

Данные элементарных типов представляют собой единое и неделимое целое. В каждый момент времени они могут принимать только одно значение. Набор элементарных типов в разных языках программирования несколько различаются, однако есть типы, которые поддерживаются практически везде. Рассмотрим их на примере языка Паскаль:

```
var
  i, j: integer;
  x: real;
  s: char;
  b: boolean;
  p: pointer;
```

Здесь объявлены переменные *i*, *j* целочисленного типа, *x* – вещественного, *s* – символьного, *b* – логического типа и *p* – указатель.

К данным элементарных типов можно обращаться по их именам:

```
i := 46;

x := 3.14;

s := ’A’;

b := true;
```

1.1. Данные числовых типов

1.1.1. Данные целочисленного типа

С помощью целых чисел может быть представлено количество объектов, являющихся дискретными по своей природе (т. е. счетное число объектов).

Диапазон возможных значений целых типов зависит от их внутреннего представления, которое может занимать 1, 2 или 4 байта. В табл. 1 приводится перечень целых типов, размер памяти для их внутреннего представления в битах, диапазон возможных значений.

Таблица 1

Тип	Диапазон значений	Машинное представление
shortint	–128…127	8 бит со знаком
integer	–32768…32767	16 бит со знаком
longint	–2147483648…2147483647	32 бита со знаком
byte	0...255	8 бит без знака
word	0…65535	16 бит без знака
comp	$-2^{63}+1...2^{63}-1$	64 бита со знаком

1.1.2. Данные вещественного типа

В отличие от целочисленных типов, значения которых всегда представляются в памяти ЭВМ абсолютно точно, значение вещественных типов определяет число лишь с некоторой конечной точностью, зависящей от внутреннего формата вещественного числа.

Суммарное количество байтов, диапазоны допустимых значений чисел вещественных типов, а также количество значащих цифр после запятой в представлении чисел приведены в табл. 2.

Таблица 2

Тип	Диапазон значений	Значащие цифры	Размер в байтах
real	$2{,}9\cdot10^{-39}...1{,}7\cdot10^{38}$	11–12	6
single	$1{,}4\cdot10^{-45}…3{,}4\cdot10^{38}$	7–8	4
double	$4{,}9\cdot10^{-324}…1{,}8\cdot10^{308}$	15–16	8
extended	$3{,}1\cdot10^{-4944}…1{,}2\cdot10^{4932}$	19–20	10

1.1.3. Операции над данными числовых типов

Над числовыми типами, как и над всеми другими возможны, прежде всего, четыре основных операции: создание, уничтожение, выбор, обновление. Специфическими операциями над числовыми типами являются арифметические операции: сложение, вычитание, умножение, деление. Операция возведения в степень в некоторых языках также является базовой и обозначается специальным символом или комбинацией символов, в других – выполняется встроенными функциями.

Следует обратить внимание на то, что операция деления по-разному выполняется для целых и вещественных чисел. При делении целых чисел дробная часть результата отбрасывается, как бы близка к 1 она ни была. В связи с этим в языке Паскаль имеются даже разные обозначения для деления вещественных и целых чисел – операции «/» и «div» соответственно. В других языках оба вида деления обозначаются одинаково, а тип деления определяется типом операндов. Для целых операндов возможна еще одна операция – остаток от деления (в языке Паскаль операция «mod»).

Еще одна группа операций над числовыми типами – операции сравнения >, <, ≥, ≤, =. Существенно, что хотя операндами этих операций являются данные числовых типов, результат их имеет логический тип – «истина» или «ложь». Говоря об операциях сравнения, следует обратить внимание на особенность выполнения сравнений на равенство/неравенство вещественных чисел. Поскольку эти числа представляются в памяти с

некоторой (не абсолютной) точностью, сравнения их не всегда могут быть абсолютно достоверны.

Поскольку одни и те же операции допустимы для разных числовых типов, возникает проблема арифметических выражений со смешением типов. В реальных задачах выражения со смешанными типами встречаются довольно часто. Поэтому большинство языков допускает выражения, операнды которых имеют разные числовые типы, но обрабатываются такие выражения в разных языках по-разному. В одних языках все операнды выражения приводятся к одному типу, а именно к типу той переменной, в которую будет записан результат, а затем уже выражение вычисляется. В других (например, язык Си) преобразование типов выполняется в процессе вычисления выражения, при выполнении каждой отдельной операции, без учета других операций; каждая операция вычисляется с точностью самого точного участвующего в ней операнда.

1.2. Данные символьного типа

Значением символьного типа char являются символы из некоторого предопределенного множества. В качестве примеров этих множеств можно назвать ASCII (American Standard Code for Information Interchange). Это множество состоит из 256 разных символов, упорядоченных определенным образом, и содержит символы заглавных и строчных букв, цифр и других символов, включая специальные управляющие символы.

Значение символьного типа char занимает в памяти 1 байт. Код от 0 до 255 в этом байте задает один из 256 возможных символов ASCII таблицы.

ASCII включает в себя буквенные символы только латинского алфавита. Символы национальных алфавитов занимают «свободные места» в таблице кодов и, таким образом, одна таблица может поддерживать только один национальный алфавит. Этот недостаток преодолен во множестве UNICODE. В этом множестве каждый символ кодируется двумя байтами, что обеспечивает более 216 возможных кодовых комбинаций и дает возможность иметь единую таблицу кодов, включающую в себя все национальные алфавиты. UNICODE, безусловно, является перспективным, однако, повсеместный переход к двухбайтным кодам символов может вызвать

необходимость переделки значительной части существующего программного обеспечения.

Специфические операции над символьными типами – только операции сравнения. При сравнении коды символов рассматриваются как целые числа без знака. Кодовые таблицы строятся так, что результаты сравнения подчиняются лексикографическим правилам: символы, занимающие в алфавите места с меньшими номерами, имеют меньшие коды, чем символы, занимающие места с большими номерами.

1.3. Данные логического типа

Значениями логического типа может быть одна из предварительно объявленных констант false (ложь) или true (истина).

Данные логического типа занимают один байт памяти. При этом значению false соответствует нулевое значение байта, а значению true – любое ненулевое значение байта.

Над логическими типами возможны операции булевой алгебры – НЕ (not), ИЛИ (or), И (and), ИСКЛЮЧАЮЩЕЕ ИЛИ (xor). Последняя операция реализована для логического типа не во всех языках.

Кроме того, следует помнить, что результаты логического типа получаются при сравнении данных любых типов.

Интересно, что в языке Си данные логического типа отсутствуют, их функции выполняют данные числовых типов, чаще всего типа int. В логических выражениях операнд любого числового типа, имеющий нулевое значение, рассматривается как «ложь», а ненулевое – как «истина». Результатами выражений логического типа являются целые числа 0 (ложь) или 1 (истина).

1.4. Данные типа указатель

Тип указателя представляет собой адрес ячейки памяти. Физическое представление адреса существенно зависит от аппаратной архитектуры вычислительной системы.

В языках программирования высокого уровня определена специальная константа nil, которая означает пустой указатель или указатель, не содержащий какой-либо конкретный адрес.

При решении прикладных задач с использованием языков высокого уровня наиболее частые случаи, когда могут понадобиться указатели, следующие:

1) при необходимости представить одну и ту же область памяти, а следовательно, одни и те же физические данные как данные разной логической структуры. В этом случае вводятся два или более указателей, которые содержат адрес одной и той же области памяти, но имеют разный тип. Обращаясь к этой области памяти по тому или иному указателю, можно обрабатывать ее содержимое как данные того или иного типа;

2) при работе с динамическими структурами данных. Память под такие структуры выделяется в ходе выполнения программы, стандартные процедуры/функции выделения памяти возвращают адрес выделенной области памяти – указатель на нее. К содержимому динамически выделенной области памяти можно обращаться только через такой указатель.

В языках высокого уровня указатели могут быть типизированными и нетипизированными. При объявлении типизированного указателя определяется и тип данного в памяти, адресуемого этим указателем. Приведем пример объявления в языке Паскаль различных типизированных указателей:

```
var
  ipt: ^integer;
  cpt: ^char;
```

Здесь переменная ipt содержит адрес области памяти, в которой хранится целое число, а cpt – адрес области памяти, в которой хранится символ. Хотя физическая структура адреса не зависит от типа и значения данных, хранящихся по этому адресу, считается, что указатели ipt и cpt имеют разный тип.

Нетипизированный указатель служит для представления адреса, по которому содержатся данные неизвестного типа. В Паскале это тип pointer. Работа с нетипизированными указателями существенно ограничена, они могут использоваться только для сохранения адреса, а обращение по адресу, задаваемому нетипизированным указателем, невозможно.

Основными операциями, в которых участвуют указатели, являются *присваивание, получение адреса, выборка*.

Присваивание является двухместной операцией, оба операнда которой указатели. Как и для других типов, операция присваивания копирует значение одного указателя в другой, в результате оба указателя будут содержать один и тот же адрес памяти. Если оба указателя, участвующие в операции присваивания типизированные, то оба они должны указывать на данные одного и того же типа.

Операция *получения адреса* – одноместная, ее операнд может иметь любой тип. Результатом является типизированный (в соответствии с типом операнда) указатель, содержащий адрес операнда.

Операция *выборки* – одноместная, ее операндом является типизированный указатель. Результатом этой операции являются данные, выбранные из памяти по адресу, заданному операндом. Тип результата определяется типом указателя.

Перечисленных операций достаточно для решения задач прикладного программирования, поэтому набор операций над указателями, допустимых в языке Паскаль, этим и ограничивается. Системное программирование требует более гибкой работы с адресами, поэтому, например, в языке Си доступны также операции адресной арифметики.